Chem 282B

HPC Regression Platform
Final Project Report

Team DOW-1-26

Kayleen Nordyke
Shree Patel
Shawn Schulz
Brent Thorne

May 2, 2026

Executive Summary

High-performance computing (HPC) environments change constantly—compilers, libraries, hardware, and policies can shift in ways that silently alter scientific results. Without a repeatable way to run representative workloads and compare them with a trusted baseline, teams risk making decisions based on outdated assumptions. This project closes that gap with an HPC Regression Platform that makes validation a routine and auditable engineering practice instead of an ad hoc effort.

The platform provides a structured path from defined workloads to metrics, stored results, and historical comparisons. It supports CLI, API, and dashboard interfaces so that automation and interactive users share the same regression workflow. The system emphasizes solver-first configuration, consistent naming, and clear separation between workload scripts and orchestration. These practices keep the platform portable across heterogeneous HPC environments while preserving a stable identity for each run.

Documentation, testing, and verification are treated as first-class deliverables. Continuous integration, layered tests, and clear architectural contracts keep changes trustworthy and demonstrations representative of real system behavior. The result is a maintainable and transparent foundation that reduces operational risk for shared clusters and positions Dow to make regression testing a standard practice.

Contents

1 Technical Overview **3**

1.1 System Context and Requirements **3**

1.1.1 Problem and Motivation 3

1.1.2 Stakeholders, Outcomes, and Constraints 3

1.1.3 High-Level System Framing 4

1.2 Architecture, Design, and Implementation **5**

1.2.1 Architecture Overview 5

1.2.2 Design Philosophy and Tradeoffs 6

1.2.3 Implemented Capabilities and Codebase 7

1.3 Validation, Results, and Technical Challenges **8**

1.3.1 System Results and Validation 8

1.3.2 Limitations and Requirements Alignment 8

1.3.3 Technical Challenges and Future Work 9

2 Project Management Overview **10**

2.1 Planning, Structure, and Workflow **10**

2.1.1 Project Structure and Phases 10

2.1.2 Planning Tools 11

2.1.3 Tracking Practices 13

2.1.4 Execution Workflow and Schedule Performance 13

2.2 Team Dynamics, Leadership, and Communication **15**

2.2.1 Coordination and Communication 15

2.2.2 Decision Making and Leadership 15

2.2.3 Workload Distribution 15

2.3 Conclusion **16**

2.3.1 System Outcomes 16

2.3.2 Team Reflection 16

2.3.3 Project Closeout and Delivery 16

Part 1

Technical Overview

1.1 System Context and Requirements

1.1.1 Problem and Motivation

The Problem

Industrial R&D depends on computational workflows that run on shared HPC systems whose software stacks and hardware configurations evolve continuously. Even small changes in compilers, libraries, or node architectures can alter performance characteristics or scientific results. Without a structured way to rerun workloads and compare outcomes across versions, teams rely on informal checks that are inconsistent, difficult to reproduce, and hard to audit. This undermines confidence in the correctness, reproducibility, and long-term reliability of scientific workflows.

Regression Testing

Regression testing provides a systematic method for detecting performance drift and behavioral changes in scientific codes. Canonical workloads are executed under controlled conditions, and their outputs, logs, and metrics are captured for comparison against historical baselines. This discipline enables early identification of deviations, supports validation of system updates, and helps maintain trust in evolving HPC environments.

1.1.2 Stakeholders, Outcomes, and Constraints

Sponsor Context

Dow requires a dependable mechanism for verifying that shared clusters and evolving software stacks continue to behave as expected. This includes consistent workload definitions, durable storage of historical runs, and the ability to compare results across time and across

system changes. Any solution must integrate cleanly with existing HPC practices while remaining portable across heterogeneous environments.

Business and Technical Outcomes

The platform is expected to provide reliable, reviewable comparisons of solver behavior over time so that system changes can be evaluated against established baselines. To support this, runs must follow consistent naming, storage, and execution conventions that prevent fragmentation across teams or sites. Users also need clear visibility into what ran, how long it took, what was logged, and whether it succeeded, including for long-running or scheduler-backed jobs. The system must be accessible to both automated workflows and interactive users, offering a unified experience across the API, CLI, and dashboard. Finally, the platform must operate across heterogeneous HPC environments and adopt an open-source license that encourages community contributions and broader adoption.

Constraints

The sponsor expects regression discipline applied to real, industry-tested HPC workflows. The capstone project also requires credible software engineering practices: clear architectural boundaries, automated verification, and comprehensive documentation. Optional SLURM integration is supported and documented but not assumed for all developer environments.

1.1.3 High-Level System Framing

Solution Architecture

The project delivers an MVP (Minimum Viable Product) HPC Regression Platform built around a solver-first packaging model. Workloads and environments are described declaratively, while solver scripts retain control over site-specific execution details. The platform provides orchestration, capture, metric extraction, storage, and review through a unified architecture. This framing establishes a maintainable and extensible foundation for regression validation on shared HPC systems, with full operational details documented in the public repository.

1.2 Architecture, Design, and Implementation

This section describes the architectural principles that shape the system and explains how the design supports reproducible execution, long-term regression visibility, and maintainable operation. Detailed module paths, API specifications, and operational instructions are documented in the public Github repository[1] and are not repeated here.

1.2.1 Architecture Overview

High-Level Architecture

Figure 1 shows how the CLI, REST API, and dashboard interact with the core harness. The harness loads configuration, executes solver subprocesses, parses output, and persists results. Solver scripts remain the execution entrypoints, while the harness provides orchestration and capture, ensuring that workloads run consistently across environments. See `README.md`[2] and `docs/ARCHITECTURE.md`[3] in the repository for full detail.

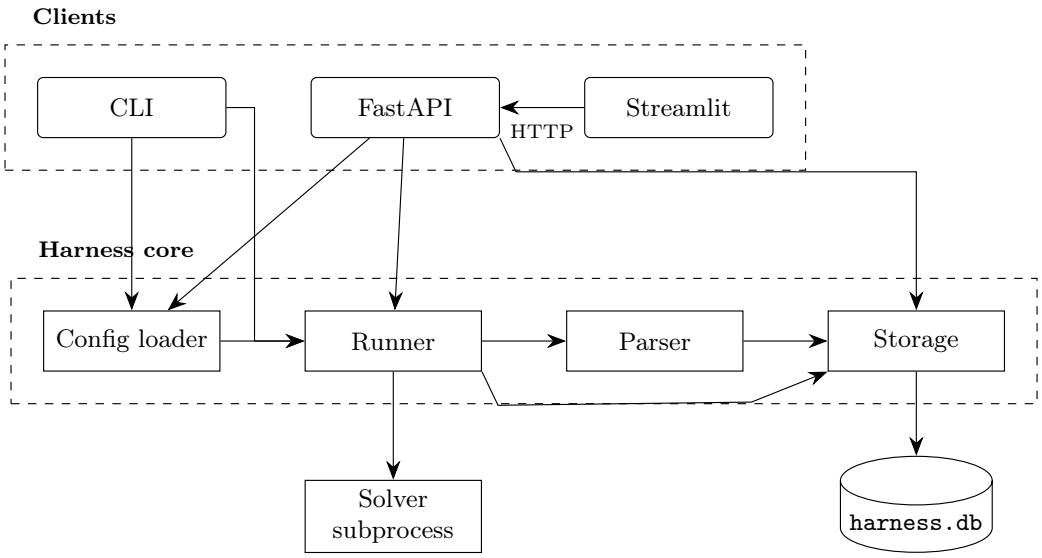

Figure 1: High-level component flow.

[1] https://github.com/fractalclockwork/hpc-runner
[2] https://github.com/fractalclockwork/hpc-runner/blob/main/README.md
[3] https://github.com/fractalclockwork/hpc-runner/blob/main/docs/ARCHITECTURE.md

Architectural Layers

The system is structured into three layers with distinct responsibilities. The **core harness** manages workload definitions, execution, parsing, and storage. The **service layer** exposes these capabilities through a stable HTTP API for automation and integration. The **dashboard** provides a browser-based interface for monitoring runs, exploring results, and reviewing trends.

This separation allows automated and batch workflows to operate independently of the UI, while enabling the UI to evolve without duplicating common harness logic. Clear boundaries between layers reduce coupling and support long-term extensibility.

1.2.2 Design Philosophy and Tradeoffs

Design Approach

The MVP emphasizes simplicity, reliability, and reproducible execution over distributed orchestration or multi-node coordination. The architecture focuses on clear data flow and stable interfaces, aligning with the sponsor's need for long-term performance tracking across large, heterogeneous HPC clusters. This approach ensures that the system remains understandable, maintainable, and suitable for regression-style validation.

Execution Philosophy

HPC environments vary widely in schedulers, modules, and parallel runtimes. To remain portable, the harness treats solver entrypoints as black boxes: solver scripts own SLURM, MPI, or container logic, while the harness manages lifecycle, capture, and measurement. This division prioritizes flexibility and avoids embedding site-specific assumptions into the core system, enabling the platform to operate across diverse environments.

Design Tradeoffs

The MVP deliberately favors robustness and clarity over advanced distributed features. It uses single-process persistence and straightforward mechanisms for tracking background runs, and it omits multi-tenant deployments or distributed control-plane services. These choices keep the system accessible to administrators and scientists and ensure that regression workflows remain efficient and easy to operate. The tradeoffs reflect a focus on practical value rather than architectural complexity.

1.2.3 Implemented Capabilities and Codebase

Capabilities Delivered

The platform supports declarative definitions of workloads and environments, making resource requirements, target systems, and solver packages explicit and reviewable. This ensures that workflows are reproducible and not dependent on personal scripts or ad hoc conventions.

The harness provides consistent execution and capture of solver behavior. Solver entry-points run as sub-processes, with output, status, and metadata recorded automatically. Both interactive and long-running jobs are supported, and background execution through the API allows users to monitor progress without waiting for completion.

Metrics extraction and historical tracking are built into the system. Named metrics are parsed from solver output and stored with each run, enabling baseline comparison, trend analysis, and regression detection. These capabilities support long-term performance monitoring and align with the sponsor's operational needs.

The platform offers multiple interfaces—CLI, REST API, and dashboard—so that developers, integrators, and scientists can use the system in ways that fit their workflows. Each interface builds on the same underlying logic, avoiding duplication and reducing maintenance overhead.

Quality and Assurance

Documentation and automated verification are treated as core engineering practices. The repository includes unit tests for core logic, API tests for contract stability, and browser tests for UI flows. Continuous integration runs these tests on every change, ensuring that updates remain reliable and that regressions are detected early. This approach supports maintainability and provides confidence that the system behaves consistently as it evolves.

Repository Structure and Contents

The public GitHub repository[4] contains the complete codebase, including the harness, API service, dashboard, configuration schemas, example workloads, automated tests, and documentation. This structure ensures that the system is reproducible, reviewable, and ready for long-term maintenance and extension.

[4] `https://github.com/fractalclockwork/hpc-runner`

1.3 Validation, Results, and Technical Challenges

1.3.1 System Results and Validation

System Outcomes and Demonstration

The MVP establishes a complete and reproducible operational loop for solver driven workloads. Declarative workload definitions are executed through a consistent harness, with outputs, logs, and metrics captured and stored for historical comparison. Users can review baselines, trends, and regressions through the CLI, REST API, and dashboard, supporting both automated workflows and interactive exploration.

These capabilities show that the system reliably executes end to end workflows, preserves the information needed for long term performance tracking, and provides a unified interface for accessing results. All delivered functionality is documented and maintained in the public repository, ensuring that the system is transparent, reviewable, and ready for continued use.

System Validation

Validation relied on automated and repeatable checks. Continuous integration runs unit tests, API contract tests, and browser level UI tests to ensure that changes do not introduce regressions. Sponsor reviews and demonstrations reflect actual system behavior exercised through the same interfaces used by developers and integrators. Together, these practices confirm that the platform is stable, predictable, and suitable for ongoing operational use.

1.3.2 Limitations and Requirements Alignment

Scope and Limitations

The MVP focuses on the core functions required for regression testing in scientific HPC workflows. It does not attempt to provide generalized cluster orchestration, distributed execution management, or multi tenant scheduling. These capabilities fall outside the intended scope. Instead, the system concentrates on areas that directly support the sponsor's operational needs, such as reproducibility, performance tracking, and long term visibility into solver behavior.

Alignment with Sponsor Requirements

The platform aligns with the sponsor's need to reduce operational risk on shared clusters and evolving software stacks. It provides canonical workloads, consistent metrics, historical comparisons, and clear visibility into performance drift. These capabilities support informed decision making for system administrators and workflow owners and establish a foundation for long term regression monitoring across heterogeneous HPC environments.

1.3.3 Technical Challenges and Future Work

Execution and Portability Challenges

HPC sites differ in schedulers, module systems, environment hierarchies, and local policies. Embedding assumptions from any single cluster would limit portability. To avoid this, the system separates configuration from execution logic.

Site specific details are captured in YAML configuration files. These files define resources, module requirements, and solver related environment variables in a declarative format. The harness loads this information, validates it, and exposes the corresponding environment to the solver without embedding cluster specific behavior.

Solver scripts then own all execution logic that depends on the site. They interpret the environment variables defined in YAML, load required modules, and call the appropriate scheduler or runtime commands. This keeps the harness neutral with respect to execution while allowing solvers to adapt to local policies and tooling.

This category also includes configuration drift and timing consistency. Schemas define the authoritative structure for workloads, and refinements are documented with migration notes to maintain alignment. Timing is taken from solver emitted values when available and supplemented with scheduler metadata when needed. These practices support reproducibility across heterogeneous environments.

Interface and Workflow Coherence

Early prototypes separated job execution from solver management, which created duplicated mental models and unnecessary UI complexity. The design was consolidated around a small number of primary workflows, including matrix style workload selection and a unified activity view.

A single service contract shared across the CLI, API, and UI ensures consistent behavior across interfaces. The dashboard acts strictly as a client of the HTTP API, which prevents divergence between interactive and automated workflows and maintains trust in the system's

behavior.

Future Work

Future development naturally extends toward enterprise hardening. Potential directions include multi worker API support, shared database backends, stronger tenancy models, and more granular access control. The MVP provides a stable foundation for this evolution by establishing clear interfaces, consistent data models, and a portable execution model.

Part 2

Project Management Overview

2.1 Planning, Structure, and Workflow

The project followed a phased plan supported by flexible workstreams. This structure guided early architectural decisions while allowing the team to adapt to sponsor feedback, integration constraints, and evolving requirements. The subsections below summarize how the project was organized, how planning tools supported execution, and how workflow practices shaped day to day progress.

2.1.1 Project Structure and Phases

Project Phases and Workstreams

The project was organized around five major phases that provided sequencing and structure. Workstreams operated in parallel within these phases and offered flexible areas of focus rather than strict ownership boundaries. This combination allowed the team to coordinate long term goals while adapting to integration needs and sponsor priorities.

Project Phases

1. **Architecture and Schemas** Defined the system architecture and established schemas for resources, systems, solvers, and tests. This phase produced the initial specification and set the foundation for all subsequent work.

2. **Core System Development** Implemented the execution neutral runner, parser subsystem, and data model. The focus was on establishing reliable execution and capture before layering on additional functionality.

3. **Integration and UI** Built the REST API, integrated the dashboard, and connected the data layer to visualization components. This phase unified the system into a coherent end to end workflow.

4. **Testing and Validation** Executed sample solvers, validated parser accuracy, verified performance drift detection, and conducted end to end tests to ensure correctness and stability.

5. **Finalization** Refined the UI, completed documentation, and prepared sponsor facing deliverables, including the final presentation and repository structure.

Supporting Workstreams

Workstreams complemented the phased structure by providing continuity across the project. **Configuration and System Definitions** evolved alongside integration feedback as schemas were refined and stabilized. The **Runner and Orchestration** workstream progressed from a prototype to a reliable subsystem capable of consistent execution and capture. **Logging, Metrics, and the Data Layer** matured into a normalized foundation for trend analysis and regression detection. The **Dashboard and UI** workstream delivered a lightweight interface for executing tests and exploring results, with iterative improvements to clarity and usability. Finally, **Administrative and Deliverables** supported scoping, stakeholder coordination, documentation, and preparation of final artifacts. Together, these workstreams enabled parallel progress while maintaining alignment with the overall plan.

2.1.2 Planning Tools

The project began with several planning artifacts that provided structure and clarified dependencies. A Gantt chart created during initial planning mapped timelines and major milestones, offering a shared understanding of the project trajectory. Although it became difficult to maintain as priorities shifted, it remained a useful high level reference for communicating progress to stakeholders.

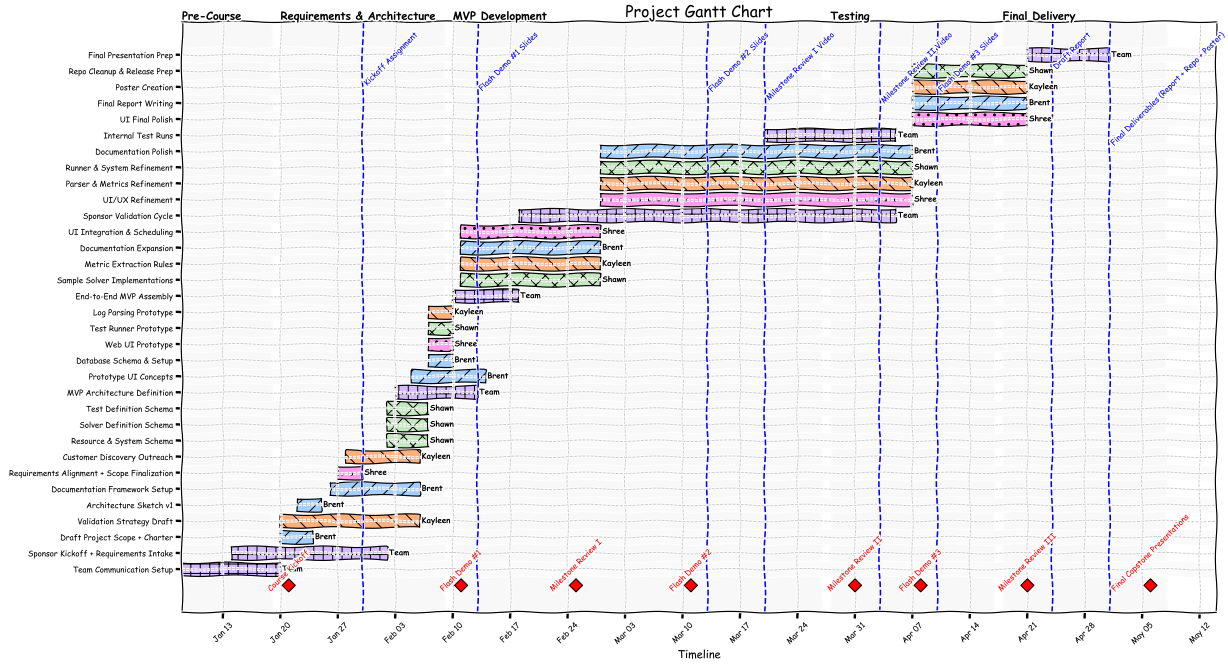

Figure 2: Gantt chart showing the project timeline and major milestones.

An activity network complemented the Gantt chart by illustrating major phases and their dependencies. This visualization clarified sequencing constraints and helped identify the critical path that determined the earliest possible completion date.

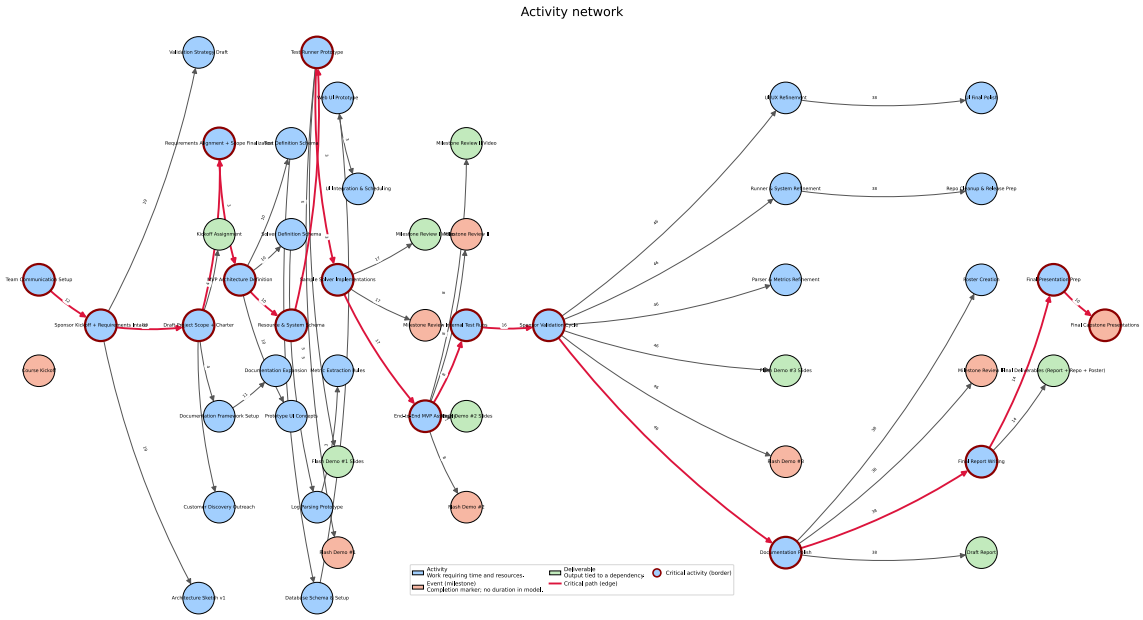

Figure 3: Activity network for major project phases and dependencies.

The critical path followed the longest chain of dependent work, beginning with team setup and requirements intake, continuing through schema design and the test runner prototype, and culminating in the end to end MVP, validation, documentation, and final presentations. This sequence defined the minimum timeline for project completion.

2.1.3 Tracking Practices

Tracking practices supported day to day continuity and informed decision making. GitHub Issues provided a structured record of tasks, ownership, and progress. Weekly notes captured decisions and next steps, while Discord enabled rapid coordination without requiring frequent meetings. Early decision logs documented architectural choices and their rationale, though they became harder to maintain as development accelerated.

Time tracking played a central role in monitoring effort. Each team member logged hours with enough detail to distinguish Capstone work from Chem 282B project management tasks. This granularity enabled the Actual Cost Adjustment, which separated total logged hours from Capstone relevant effort and provided accurate data for Earned Value analysis.

2.1.4 Execution Workflow and Schedule Performance

Project execution used a lightweight, disciplined workflow based on GitHub Issues, weekly sprints, and short standups. Early duplication of work and unclear ownership led to explicit issue assignment and a clearer division of responsibilities, improving coordination, reducing integration friction, and standardizing feature and bug-fix work through a pull-request review pipeline.

Time tracking improved schedule visibility. By separating non-Capstone hours, the team aligned Actual Cost (AC) and Earned Value (EV) to the same scope, eliminating distortions from other academic work and providing clean inputs for Earned Value Management.

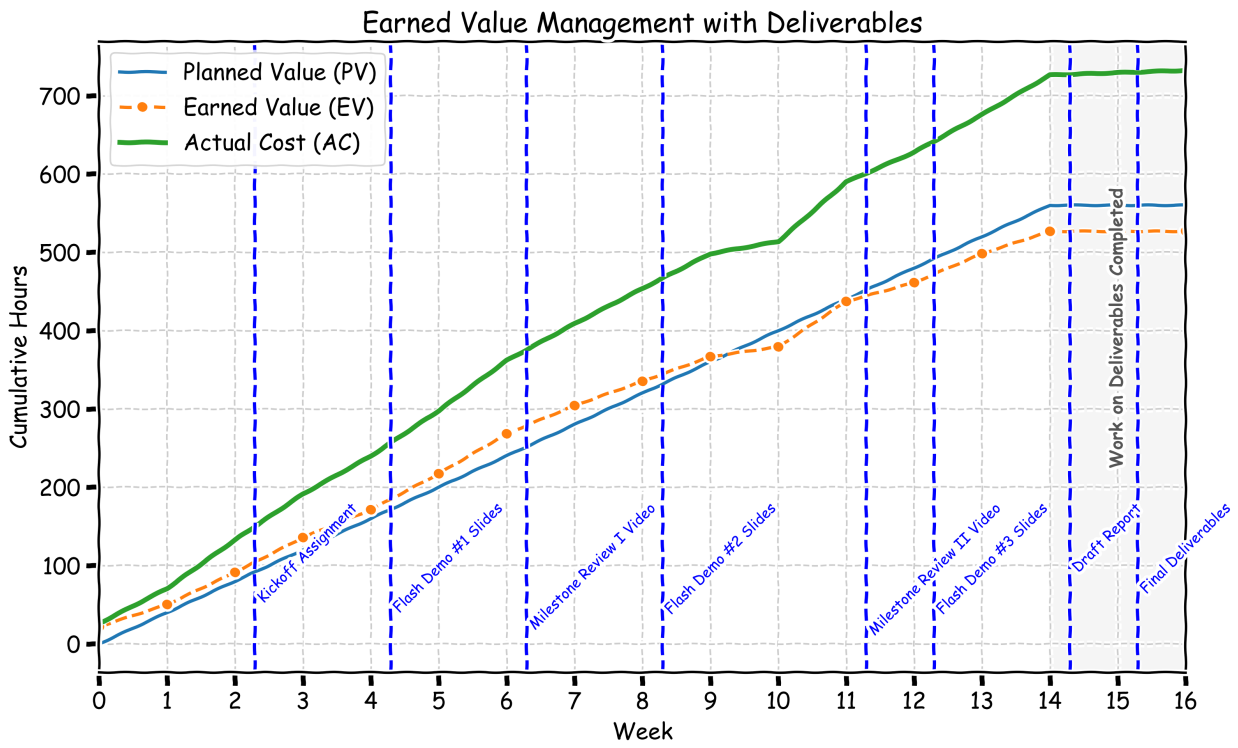

Figure 4: Earned Value Management chart showing Planned Value (PV), Actual Cost (AC), Earned Value (EV), and deliverable deadlines. Weeks 14–16 are shaded to indicate completion of all deliverables and the end of recorded Capstone work.

From a PMBOK[5] perspective, the Earned Value trends in Figure 4 show a well-controlled project. EV tracked PV closely through Week 14, indicating near-planned delivery and a Schedule Performance Index near 1.0 despite normal availability fluctuations. The flattening of PV after Week 14 marks scheduled completion, and the shaded region shows only minimal follow-on effort.

The gap between AC and EV reflects exclusion of non-Capstone hours and gives a consistent view of cost versus progress. Because AC includes reclassified, out-of-scope hours, the Cost Performance Index is intentionally conservative, aligning cost and value to the same scope baseline and avoiding overstated performance. In PMBOK terms, the team controlled both schedule and cost baselines by keeping all measurements aligned with defined scope.

Overall, the project shows disciplined PMBOK schedule and cost control. The team maintained steady momentum, delivered value at a reliable rate, and produced a clear, data-driven performance assessment across the semester.

[5]PMBOK: Project Management Body of Knowledge, the Project Management Institute's standard framework for project management processes and performance measurement.

2.2 Team Dynamics, Leadership, and Communication

2.2.1 Coordination and Communication

The team's coordination practices strengthened steadily over the semester. Early meetings were structured and productive, and the team quickly established a collaborative rhythm supported by clear communication channels. As the project matured, the team refined its workflow by introducing clearer deadlines, consistent meeting formats, and explicit expectations, which enhanced alignment and improved visibility into shared progress. Collaboration relied on GitHub for structured issue tracking and code review, and on Discord for rapid coordination and real-time problem solving. This combination of asynchronous updates and synchronous discussion enabled the team to balance availability constraints while maintaining continuous forward momentum.

2.2.2 Decision Making and Leadership

Decision making was highly participative, with roles defined at a high level and task ownership tracked through GitHub Issues. The team demonstrated strong situational leadership, with members stepping forward to make timely decisions when needed to preserve momentum. As the semester progressed, the team further strengthened its decision processes by clarifying responsibilities and increasing transparency around design choices. These practices supported coherent development across interconnected components and reinforced a shared understanding of system architecture and priorities.

2.2.3 Workload Distribution

Workload distribution evolved naturally as development progressed. Team members with deeper engineering experience took on complex or time-sensitive tasks, ensuring that critical work advanced efficiently. At the same time, cross-learning efforts broadened skills across the team and deepened collective understanding of the system. The team proactively clarified ownership of key subsystems, coordinated closely in standups, and adjusted expectations as needed to maintain balance. These practices enabled the team to operate effectively within a mixed-experience environment while sustaining consistent progress throughout the semester.

2.3 Conclusion

2.3.1 System Outcomes

The project concludes with a stable and coherent MVP that meets the sponsor's core requirements and provides a solid foundation for long term regression testing on shared HPC systems. The platform supports reproducible runs, consistent metric collection, and durable historical comparisons, demonstrating that the architecture, data model, and execution pipeline operate as an integrated whole. Iterative refinement produced a reliable operational loop that detects performance drift and verifies solver behavior across diverse environments. The final system is maintainable, extensible, and aligned with Dow's long term goals.

2.3.2 Team Reflection

The team gained significant technical and organizational experience. Early architectural alignment, clear interfaces, and disciplined implementation proved essential for a system with many interdependent components. The project also highlighted the importance of steady coordination during uneven contribution periods and prompted improvements to communication and ownership. These lessons strengthened the team's technical judgment, collaboration skills, and ability to manage integration challenges and shifting priorities. The emphasis on documentation, clarity, and cross functional problem solving will support future academic and industrial work.

2.3.3 Project Closeout and Delivery

The final deliverable was provided to Dow through a public GitHub repository at `https://github.com/fractalclockwork/hpc-runner`. The repository includes all source code, usage instructions, architecture and design documentation, example solvers, Docker environments used for development and validation, and automation scripts for running workloads and collecting metrics. These materials form a complete and self contained handoff that enables Dow to install, operate, and extend the system without additional dependencies.